Learning an identity mapping through three hidden units

ACML Assignment 1 | Manual backpropagation | 8-3-8 autoencoder

Authors: Thai Binh Nguye and Ege Deniz

1. Task and runnable software

The network learns the identity mapping on all eight rows of the 8 x 8 identity matrix. Each input has one active position, and its target is the same vector. Both hidden and output layers use sigmoid activations. All 59 parameters (24 + 3 + 24 + 8) are trained. NumPy supplies array operations; forward propagation, gradients and updates are implemented manually. No neural-network framework or automatic differentiation is used.

From the code directory, install dependencies with `python -m pip install -r requirements.txt`. Run `python nn.py` for the demo and gradient check; `python experiments.py` reproduces all 80 experimental runs and figures. Open `lab1.ipynb` for a guided walkthrough. `python build_report.py` rebuilds this report after the experiments. The input data are generated by np.eye(8), so no external dataset is required.

2. Forward pass, objective and backpropagation

Examples are rows: X has shape (m,8), W1 is (8,3), b1 is (3,), W2 is (3,8), and b2 is (8,). Biases broadcast over examples. The forward pass is:

```
H = sigmoid(X @ W1 + b1)
Y = sigmoid(H @ W2 + b2)
L = sum((Y - X)**2) / (2*m)
```

The loss averages over examples, and sums over the eight outputs. It is not the mean over all 64 entries. Since sigmoid derivative is a(1-a), the chain rule gives:

```
D2 = (Y-X) * Y * (1-Y) / m
D1 = (D2 @ W2.T) * H * (1-H)
dW2 = H.T @ D2       db2 = sum(D2, axis=0)
dW1 = X.T @ D1       db1 = sum(D1, axis=0)
parameter = parameter - learning_rate * gradient
```

The output error is propagated through W2 to determine each hidden unit's contribution. The hidden activation derivative is essential. Bias gradients are the sum of the corresponding deltas. All gradients use the pre-update parameters; only then are weights and biases updated together. Training uses full-batch gradient descent: one epoch presents all eight patterns once.

Weights are sampled independently from a uniform distribution with bounds +/- sqrt(6/11), multiplied by the initialization scale; biases start at zero. Random weights break hidden-unit symmetry. We use neither regularization nor momentum. Compared with Backprop Help, our row-vector notation transposes the weight convention, and its optional weight-decay term is omitted.

3. Verification and learning experiments

Every one of the 59 analytic parameter derivatives was compared with a central finite difference (epsilon = 1e-5) on three distinct examples at seed 7. Maximum absolute errors were:

W1: 1.434e-11; **b1**: 8.512e-12; **W2**: 1.029e-11; **b2**: 4.226e-12.

Success means every output on every pattern is within 0.10 of its target. This is stronger than correct argmax or thresholded reconstruction. Runs stop at the first successful epoch or at 20,000 epochs. For each configuration we used seeds 0-9. Learning-rate experiments fix initialization scale at 1; initialization experiments fix learning rate at 3. Failed runs are retained in the results.

Rate	Init. scale	Success	Median epochs*	Median seconds**	Median loss
0.1	1	0/10	-	1.565	0.08095
1	1	10/10	10,099	0.825	0.00661
3	1	10/10	3,376	0.265	0.00659
10	1	10/10	1,192	0.099	0.00614
100	1	0/10	-	1.521	0.43750
3	0	0/10	-	1.589	0.05338
3	0.01	10/10	4,360	0.314	0.00629
3	5	10/10	3,658	0.236	0.00619

* Epoch median includes successful runs only. ** Time median includes all runs, including failures at the cap. Timing is local wall-clock time and is hardware/load dependent (3.12.13, NumPy 2.5.4, Windows). A failed run means failure within the fixed budget, not proof that eventual convergence is impossible.

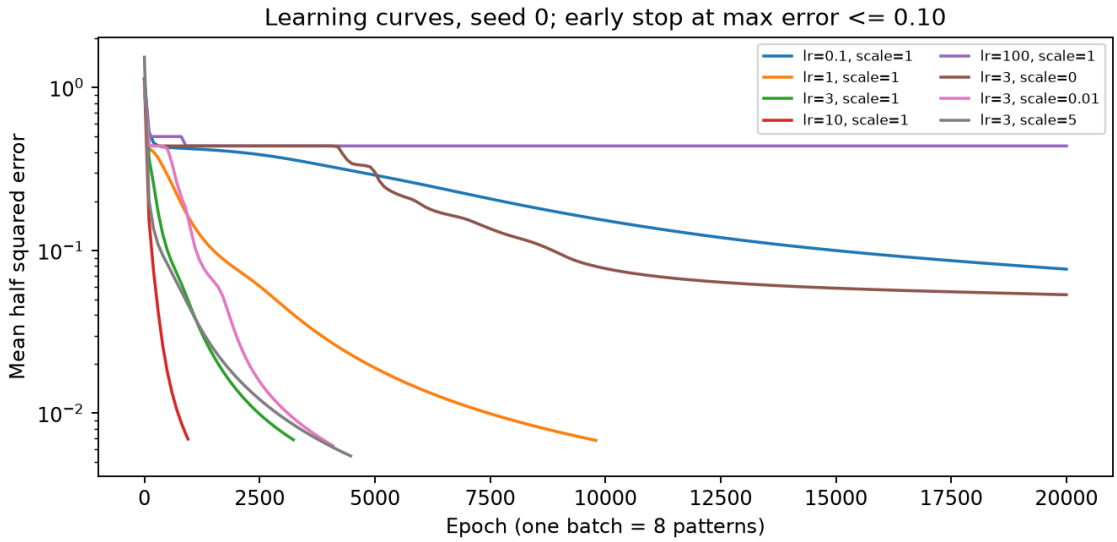


Figure 1. Loss trajectories for seed 0. Curves end early when the stopping criterion is met. The same seeds across settings provide paired initialization comparisons.

Moderate learning rates converge faster as the rate increases in the tested range. Rate 0.1 is too slow for the chosen budget; rate 100 fails to meet the criterion, showing that a larger step is not always better. Zero initialization preserves symmetry, preventing distinct position codes. Very small initialization delays symmetry breaking, while large initialization can saturate sigmoid units and reduce useful gradients. The zero-scale runs start from identical parameters despite different seeds, so they repeat one deterministic control. The success counts quantify dependence on initialization; ten seeds give evidence for these settings, not a guarantee for every random start.

4. Reconstruction and interpretation of the hidden layer

The representative run was fixed in advance: seed 0, learning rate 3, scale 1. It reached the criterion after 3,233 epochs, or 25,864 pattern presentations (0.239 seconds). Its final loss was 0.006869, maximum absolute error 0.099993, and all 8/8 patterns were reconstructed exactly after thresholding at 0.5.

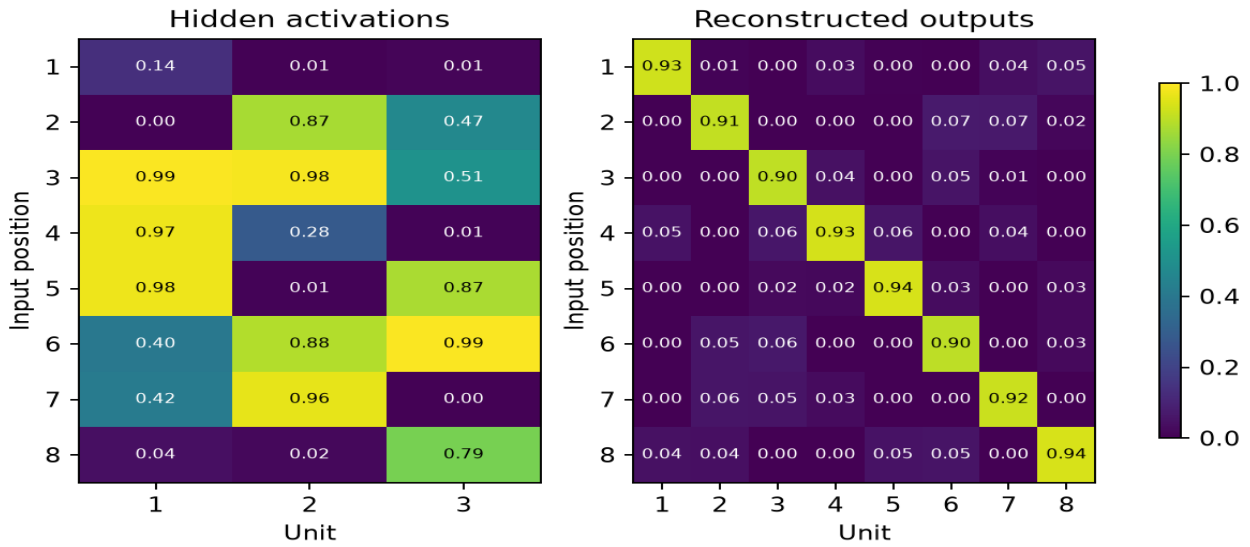


Figure 2. Actual sigmoid values, not thresholded values. Rows correspond to the active input position. The output diagonal is high and off-diagonal entries are low, demonstrating reconstruction of each training pattern.

Position	Hidden H1	Hidden H2	Hidden H3	Threshold code
1	0.136	0.007	0.012	000
2	0.005	0.869	0.467	010
3	0.994	0.985	0.511	111
4	0.973	0.285	0.009	100
5	0.976	0.009	0.871	101
6	0.400	0.884	0.993	011
7	0.415	0.962	0.004	010
8	0.039	0.020	0.793	001

Hidden units encode groups of positions rather than duplicating the eight input nodes. In this run H1 is above 0.5 for positions 3, 4 and 5; H2 for 2, 3, 6 and 7; H3 for 3, 5, 6 and 8. Positions 2 and 7 share threshold code 010, but have different continuous activations, especially H3 (0.467 versus 0.004). There are seven distinct threshold codes and eight distinct continuous representations. Three binary units have capacity for eight distinct codes, but this network does not learn a perfect binary code: the decoder uses actual continuous values. The precise assignment of codes is not prescribed and may change with random initialization.

For one-hot input e_i, hidden preactivation j equals W1[i,j] + b1[j]. Thus positive values activate a detector and negative values suppress it. For output k, the decoder computes sigmoid(sum_j H[j]*W2[j,k] + b2[k]); it combines positive and negative evidence from the compact code. Weight signs alone must be interpreted together with biases. Raw weights, biases, activations and outputs are saved in results/baseline.npz and printed in the notebook.

5. Scope and sources

All eight possible training patterns are used; this is a reconstruction demonstration, not a test of generalization to unseen mixtures or noisy inputs. Finite sigmoid weights approximate zeros and ones; the numeric outputs should not be described as exactly binary. Sources: supplied Lab - NN.pdf (architecture and deliverables) and Backprop Help.pdf, sections 2.1-2.3 (chain-rule derivation and gradient checking).